FIG. 14A — PORTABLE ZERO-KNOWLEDGE CREDENTIAL — ARCHITECTURE & FORMATS 1400

SECTION A: TWO-TIER CREDENTIAL ARCHITECTURE 1400

Member submits ZK credential proof (mint/holding/holdingByType) --> Sequencer verifies via CredentialProgram.verify() (Pickles recursive verification) --> issues credential tier(s):

```
TIER 1: Full ZK
PortableCredential

Size: 10-22 KB
Transport: QR/network
Trust: Mathematical
(ZK proof)
```

```
TIER 2: Ed25519
NfcCredentialAttest.

Size: ~500 bytes
Transport: NTAG216
Trust: Ed25519 sig
(sequencer receipt)
```

```
TIER 3: Delegation
PortableDelegation
Credential
Size: ~2 KB
Transport: QR / NFC
Trust: ZK      TTL
tiers
```

SECTION B: PORTABLECREDENTIAL FORMAT (FULL ZK PROOF) 1402

```
PortableCredential — 10-22 KB
version: 1
proofType: 'credential-holding'
proofJson: {
  publicInput: [...]
  publicOutput: [...]
  maxProofsVerified: 1
  proof: "KChp...base64" (10-22KB)
}
```

```
publicOutput: {
  walletId: "48291637..."
  operationType: "2" (holding)
  credentialCommitment: "73829164..."
  oldInventoryRoot: "55182736..."
  newInventoryRoot: "55182736..."
}
verificationKeyHash: "91638274..."
createdAt / expiresAt (24h default)
```

THREE CREDENTIALPROGRAM METHODS (ZkProgram):

```
METHOD 1: mint
Proves correct credential insertion
  into wallet inventory Merkle tree
Input: Credential    +    old inv root
Output: walletId, credCommitment,
  oldInvRoot -> newInvRoot
In-circuit: recursive wallet verify
```

```
METHOD 2: holding
Proves wallet holds SPECIFIC
  credential via Merkle inclusion
Input: Credential    +    Merkle witness
Output: walletId, credCommitment
In-circuit: inclusion    +    recursive
  wallet proof
```

```
METHOD 3:
holdingByType
Proves wallet
  holds cred of
  SPECIFIED TYPE
  without which
SELECTIVE
DISCLOSURE
```

SECTION C: NFCCREDENTIALATTESTATION FORMAT (COMPACT) 1404

```
NfcCredentialAttestation — ~500 bytes

v: 2 (format version)
t: 'credential-attestation'
w: "48291637..." (wallet ID)
cc: "73829164..." (cred commitment)
ls: 3 (lifecycle = ACTIVE)
fr: "91827364..." (forest root)
```

```
ts: 1709312000 (Unix seconds)
exp: 1709398400 (    +   24h default)
n: "a1b2c3d4..." (revocation nonce, 32 hex)

pk: "7f8e9d0c..." (Ed25519 pubkey, 64 hex)
sig: "3a4b5c6d..." (Ed25519 signature, 128 hex)
```

Canonical sign message (pipe-delimited, deterministic):
  "zkpcare-attestation-v2|{w}|{cc}|{ls}|{fr}|{ts}|{exp}|{n}"
  SIGN: sig = Ed25519.sign(canonicalMessage, privateKey)
  VERIFY: ok = Ed25519.verify(canonicalMessage, publicKey, sig) — no JSON ambiguity

SequencerAttestationAuthority:
  Construction: random keypair OR deterministic from seed (SHA-256 -> PKCS8 DER -> Ed25519)
  Methods: createAttestation(params), verifyAttestation(att) [STATIC], revokeNonce(n), isRevoked(n)